

Sparse Certified Port Incidence for Compressed Unknot-Recognition Pipelines

Output-sensitive reconstruction, polynomial support bounds,
and exact coning continuations

Research continuation for the ProveIt project

Prepared by ChatGPT for Vladimir Reshetnikov

8 October 2026

Abstract

The maintained marked-orbit interface in ProveIt reconstructs component incidence with r ports by a dense Boolean inversion with 2^r output slots. We replace this representation with exact sparse recovery from the same unweighted orbit-count primitive. For s occurring signatures, balanced block deletion uses $O(s + \sum_{\emptyset \neq T \in \text{supp}_h} |T| \log(1 + r/|T|))$ logical queries, without knowing s in advance. An independent certificate needs at most $s + \sum_T |T|$ authenticated subset counts, plus the baseline count.

For interval-encoded ports, sparsity need not be assumed: a run-accounting consequence of the classical weighted Agol–Hass–Thurston algorithm bounds s by $O(M + (k + 1)^3(B + 1))$, where k is the number of pairings, M the number of marked intervals, and B the endpoint bit length. Thus the new black-box reduction is polynomial in its local encoded input, with no logarithmic restriction on the number of ports. Polynomial weighted orbit computation itself is classical, not a novelty claim of this note.

We also prove exact updates under coning and union re-marking, and refine incidence by a supplied binary orientation character using only one further query per occurring signature. Executable standard-library Python, adversarial checks, an opt-in maintained-kernel adapter, and controlled component measurements accompany the proofs. Sparse examples improve substantially; a full-support control slows down. The measurements use a clearly labeled audit extraction, not the complete maintained recognizer. These results remove a local exponential representation bottleneck; they do not establish an end-to-end quasi-polynomial unknot-recognition algorithm.

Status. Mathematical arguments and executable research artifacts; not a formally verified or peer-reviewed theorem package. No repository writes or production-default changes are included.

Contents

1	The bottleneck and the scope of the improvement	3
2	Interval relations and the coning oracle	4
3	Exact sparse inversion by positive residuals	5
4	Balanced block deletion and its query complexity	6
5	Short independent certificates	7
6	Why interval-encoded incidence has polynomial support	9
7	Bit costs, storage, and resource semantics	10
8	Incidence-resolved binary orientation data	11
9	Exact continuation operations and a sharp limitation	12
10	Implementation, trust boundaries, and integration	13
11	Validation and controlled component experiments	15
12	Consequences for the research program	17
13	Further research questions and concrete next experiments	18
A	A worked sparse certificate	19
B	Reproduction and artifact inventory	20

1 The bottleneck and the scope of the improvement

1.1 What is being continued

This note continues the interval-orbit branch of `Topology/UnknotRecognition`, rather than restarting the project's Khovanov, braid, determinant, or group-presentation work. The inspected integration commit for the dense incidence module is `0441aad3967b69621edabfdb85367d465b4fb32f`. This is a baseline integration commit, not an assertion about the repository's latest overall head. The exact inspected file hashes are recorded in the package's `provenance.json`.

The relevant maintained article, `synthesis/orbit_certificates.tex`, describes independently replayable unweighted orbit counts, coning a union of marked intervals, exact caching of equal physical unions, and a dense incidence histogram obtained by Boolean Möbius inversion [5, 6]. Query reuse reduces repeated work but leaves an array of length 2^r . Its conditional polynomial and quasi-polynomial bounds consequently restrict the number of ports. The interface reports properties of supplied interval relations, not a knot verdict; ordinary recognition does not currently dispatch through it [6].

The precise target here is narrower and more actionable than replacing the whole recognizer: retain the existing count/proof boundary, but eliminate unnecessary enumeration of absent incidence signatures. This separation matters. A local theorem can be fully unconditional in its encoded input while its relevance to a global algorithm still depends on how those inputs are generated.

1.2 Established results, new deductions, and integration work

The classical ingredients are the unweighted and weighted orbit algorithms of Agol, Hass, and Thurston (AHT) [1]. Their Theorem 12 supplies a polynomial unweighted count. Their weighted transfer lemma and Theorem 16 supply polynomial compressed orbit-weight data. We do not claim the first polynomial algorithm for marked or weighted components.

The contributions developed here are the following precise combination. First, a nonnegative sparse inversion can reuse only unweighted coning queries, including their existing independent proofs. Second, short minimal-support certificates avoid both dense inversion and replay of the search schedule. Third, an explicit accounting argument connects interval encoding to polynomially many occurring signatures. Fourth, a restricted continuation language and a known-support double-cover calculation give additional exact operations on the recovered representation. These are proved below; priority beyond the inspected repository is not asserted.

The supplied implementation has two distinct assurance levels. Its generic algebra, interval oracle, audit kernel, continuation operations, and signed refinement have executed tests. The adapter for the maintained modules has executed contract tests with independent literal witnesses. A genuine maintained AHT trace run is supplied as a separate integration script but was not executed in this delivery environment. No upstream full-suite pass is attributed to this package.

1.3 Relation to the quasi-polynomial goal

Lackenby's July 2026 preprint develops incompressible-surface and hierarchy methods relevant to unknot recognition. Section 9 explicitly separates the number of hierarchy operations from the still-unspecified costs of some operations, and leaves hierarchy-length and pattern-complexity parameters unbounded there [2]. The present note addresses neither of those global bounds. It identifies a class of aggregate component operations whose representation need not contribute an additional 2^r factor.

An improvement from $2^r \text{ poly}(\text{local input})$ to $\text{poly}(\text{local input})$ is useful even before a hierarchy theorem is complete. It changes which intermediate interfaces are computationally plausible. It does not show that

every relevant interface is an incidence histogram, that all future attachments preserve this information, or that a complete hierarchy has sufficiently small encoded descriptions.

2 Interval relations and the coning oracle

2.1 Input conventions

Let $X_N = \{0, 1, \dots, N-1\}$. A pairing is an isometry between two nonempty inclusive integer intervals of the same length. It is either a translation or an order reversal. The smallest equivalence relation containing all pairing arrows is denoted by $\sim$. Its classes are called *orbits*. There are k supplied pairings and $C = |X_N/\sim|$ orbits.

Ports $A_1, \dots, A_r$ are finite unions of half-open intervals $[a, b) \cap \mathbb{Z}$, with $0 \leq a \leq b \leq N$. Let M be the number of supplied marked interval records, before optional canonical merging. Empty, overlapping, coincident, and nested ports are permitted. The convention change is intentional: pairings use inclusive endpoints, while port records use half-open endpoints. All endpoints and multiplicities are binary integers. Put

$$B = \lceil \log_2(N+1) \rceil.$$

The empty universe is dealt with directly. In polynomial expressions, $B+1$ avoids degenerate zero factors.

For an orbit O , define its signature and the associated histogram by

$$T(O) = \{i \in [r] : O \cap A_i \neq \emptyset\}, \quad (1)$$

$$h(T) = |\{O : T(O) = T\}|. \quad (2)$$

Write $s = |\text{supp } h|$. The empty signature is retained. In particular, a large region missed by all ports is not silently discarded. The requested output is a list of the positive pairs $(T, h(T))$, not an array with a slot for every subset of $[r]$.

An r -bit mask occupies $O(r)$ bits, even if its numerical value is close to 2^r . Merely using masks does not allocate a truth table. Nevertheless, operations on those masks must be charged in the bit model rather than regarded as constant time for unbounded r .

2.2 Coning a physical union

For a nonempty union A of disjoint half-open intervals, add the following relations. In each interval $[a, b)$ of length at least two, add the translation $[a, b-2] \rightarrow [a+1, b-1]$. Choose one interval's first point as an anchor and join every other interval's first point to it by a singleton pairing. This construction uses at most twice the number of intervals in A and never expands an interval into points.

Lemma 2.1 (Coning identity). *Let C_A be the number of orbits after these relations are added, and let $H(A)$ be the number of original orbits meeting A . Then*

$$H(A) = C - C_A + 1.$$

For $A = \emptyset$, $H(A) = 0$ without an auxiliary query.

Proof. Every point of A is identified with the anchor. Hence every original orbit meeting A belongs to one new orbit. No orbit disjoint from A is incident to an added arrow, so it remains unchanged. Exactly $H(A)$ original orbits are replaced by one, which gives the formula. This argument applies to any initial equivalence relation; interval geometry is used only to encode the extra arrows compactly. $\square$

The identity and its interval construction are already present in the maintained module [5]. We use them as an authenticated oracle rather than changing the orbit scheduler.

For $U \subseteq [r]$, define

$$G(U) = C - H\left(\bigcup_{i \notin U} A_i\right). \quad (3)$$

An orbit contributes precisely when its signature is contained in U . Thus

$$G(U) = \sum_{T \subseteq U} h(T), \quad G([r]) = C. \quad (4)$$

When the physical complement-port union is nonempty, $G(U) = C_A - 1$; otherwise $G(U) = C$. A nonempty set of port indices may still specify an empty physical union. The implementation tests the latter, not merely whether the index mask is nonzero.

2.3 What may be cached

Canonicalize a physical union by sorting intervals and merging both overlap and adjacency. Equal canonical unions lead to the same extra pairings and the same source-bound count query. They may share a count and a proof. Equality of numerical answers alone is not a valid cache key: two different unions can have the same touched-orbit count but behave differently in other queries. In particular, the sparse reduction and physical-union caching are separate optimizations. The former removes absent output types; the latter removes duplicate oracle inputs.

3 Exact sparse inversion by positive residuals

This section applies to any nonnegative integer function on $2^{[r]}$, not just to interval systems. Its hypothesis is the full oracle contract (4). Checking a few queried values lie between zero and C does not establish that contract for an arbitrary untrusted oracle.

Suppose the algorithm has already recovered exact pairs $(T_1, w_1), \dots, (T_j, w_j)$. Define

$$R_j(U) = G(U) - \sum_{\substack{1 \leq a \leq j \\ T_a \subseteq U}} w_a. \quad (5)$$

If the previous extractions are correct, this is the zeta transform of the remaining nonnegative coefficients. Consequently it is monotone under inclusion and nonnegative on every subset.

Lemma 3.1 (Minimal positive support). *If $R_j(U) > 0$ and $R_j(U \setminus \{i\}) = 0$ for each $i \in U$, then U is an unrecovered signature and $R_j(U) = h(U)$.*

Proof. Any proper subset $S \subsetneq U$ omits some $i \in U$. Its residual coefficient is a nonnegative summand of $R_j(U \setminus \{i\})$, so it is zero. The only possible positive residual summand in $R_j(U)$ is therefore the coefficient at U itself. Positivity proves that coefficient is present; the displayed value is its entire multiplicity. $\square$

A simple extraction begins with $U = [r]$ and tries deleting each index in turn. A deletion is accepted exactly when the resulting residual remains positive. A failed deletion remains impossible after later deletions: if the candidate residual was zero, every smaller candidate also has zero residual. At the end, Lemma 3.1 applies. Importantly, one extraction removes the full multiplicity of a type, not one component. A coefficient equal to 2^{16000} therefore takes one extraction.

Theorem 3.2 (Sparse recovery). *Given C and an exact oracle satisfying (4), all positive coefficients of h can be recovered without knowing s in advance. Individual deletion uses at most sr oracle requests. The procedure stores only recovered entries and its query cache; it needs no 2^r array.*

Proof. When the remaining mass $C - \sum_{a \leq j} w_a$ is positive, the starting set $[r]$ has positive residual. Deletions preserve positivity and terminate at a set satisfying Lemma 3.1. This recovers one previously absent positive coefficient exactly and preserves the nonnegative-residual invariant. There can be at most s such extractions. Conversely, if any coefficient remained positive, the remaining mass would be positive. Termination at mass zero therefore proves completeness. There are at most r deletion probes per extraction. For $r = 0$, the only possible coefficient is $h(\emptyset) = C$ and no query is needed. $\square$

A resource cap does not alter the theorem’s acceptance condition. Exhausting a cap before mass zero means that the complete output has not been obtained. The adapter consequently returns INCONCLUSIVE, not a partial histogram labeled complete and not a negative knot certificate.

3.1 Why nonnegativity is essential

For signed coefficients, cancellation can make $R(U) = 0$ while positive and negative proper-subset coefficients remain. A deletion can then remove hidden support. For example, coefficients $h(\emptyset) = 1$ and $h(\{1\}) = -1$ have total zero but nonempty support. The mass-zero stopping rule would be invalid. Orbit multiplicities are nonnegative by definition, which supplies exactly the structure used here.

4 Balanced block deletion and its query complexity

4.1 The algorithm

Individual deletion spends a probe on every port even when a surviving type uses very few ports. Instead, arrange $[r]$ in a balanced binary partition tree. At a node with block D , query $R(U \setminus D)$. If it is positive, delete the entire block and do not visit its descendants. If it is zero, retain the block provisionally and visit both children. A singleton with a zero deletion residual is kept. The traversal is deterministic and iterative; its depth does not become a Python recursion requirement.

The algorithm’s terminal set is still inclusion-minimal positive. A retained singleton failed deletion when tested, and subsequent deletions cannot turn a zero residual into a positive one. Deleted blocks have preserved the existence of a surviving coefficient throughout.

Theorem 4.1 (Output-sensitive block bound). *Let one extraction return T , with $d = |T|$, and put $b_r = \lceil \log_2 r \rceil$ for $r \geq 1$. Its number of deletion probes is at most*

$$q_r(d) = 1 + 2 \sum_{\ell=0}^{b_r-1} \min(2^\ell, d). \quad (6)$$

In particular, $q_r(0) = 1$, and for $d > 0$ it is $O(1 + d \log(1 + r/d))$. The entire reconstruction uses at most

$$Q \leq \sum_{T \in \text{supp } h} q_r(|T|) \quad (7)$$

logical requests. For $r = 0$, set $Q = 0$.

Proof. Consider a failed block D tested at a current set U_D . If the final support T missed D , then $T \subseteq U_D \setminus D$. The coefficient at T belongs to the current extraction’s fixed residual and is positive, contradicting the failed test. Every failed block thus intersects T .

At depth ℓ , blocks are pairwise disjoint, so at most d of them can fail. There are also at most 2^ℓ blocks at that depth. Only children of failed internal nodes are queried. Counting the root probe and at most two child probes per failed internal node gives (6), because the balanced tree has height at most b_r . For $d = 0$ the root succeeds and there are no descendants. For $d > 0$, split the sum where 2^ℓ first exceeds d . The early geometric sum is $O(d)$ and the remaining $O(\log(1 + r/d))$ levels contribute at most d each. Summing over extractions proves (7). $\square$

Physical-union caching may make the number of actual orbit counts much smaller than Q . Conversely, producing independent proofs incurs further queries in the supplied two-pass implementation. These costs must not be conflated with the logical discovery bound.

4.2 A matching order of growth for a basic special case

This search is closely related to group testing. The literature on binary splitting and hypergraph reconstruction contains much broader query-model results [4, 3]; we do not claim binary splitting as a new general technique. The following elementary comparison explains why the dependence on the size of an individual signature is reasonable.

Proposition 4.2 (Singleton-profile lower bounds). *Consider deterministic algorithms correct for all singleton profiles $h(T) = 1$, with the unknown $T \subseteq [r]$. Among supports of size d , some input needs at least $\lceil \log_2 \binom{r}{d} \rceil$ oracle calls. Moreover, for every nonempty actual T , its transcript needs at least $|T|$ calls if correctness is required against supports of every size. Thus the order $d \log(1 + r/d)$ is optimal in the worst case at each $d > 0$, up to absolute constants, for this special case without a supplied rank.*

Proof. Here $G(U)$ is one exactly when $T \subseteq U$; every answer is a bit. Distinguishing $\binom{r}{d}$ possibilities gives the first decision-tree bound. For the second, compare the actual T with $T \setminus \{i\}$ for every $i \in T$. A query distinguishes this pair only when U contains all of $T \setminus \{i\}$ and omits i . One query cannot have that property for two different members of T . Every deletion alternative must be excluded, requiring at least $|T|$ calls. If $d \leq r/2$, the binomial bound is $\Omega(d \log(r/d))$; if $d > r/2$, the linear bound supplies $\Omega(d \log(1 + r/d))$. $\square$

The qualification about a supplied rank is necessary. Knowing in advance that $d = r$ determines T without any query. We also make no claim that summing the singleton lower bounds proves optimality for arbitrary multi-support profiles; that is a different adaptive reconstruction problem.

5 Short independent certificates

5.1 Certificate data and algebraic checks

A certificate lists positive pairs $(T_1, w_1), \dots, (T_s, w_s)$ in an extraction order. For each listed T_j , authenticate the following values:

$$G(T_j) \quad \text{and} \quad G(T_j \setminus \{i\}) \quad (i \in T_j). \quad (8)$$

Each authentication is either the baseline identity for an empty physical union, or an independently checked coned orbit count. Repeated physical unions share one proof. The verifier reconstructs all unions from the original ports and binds each proof to the exact original pairing list plus its deterministic coning rows.

At entry j , subtract the weights of previously listed types contained in each query mask. Require residual w_j at T_j and zero at every tested one-element deletion. Check masks are in range, types are distinct, weights are positive integers rather than Boolean values, and $\sum_j w_j = C$. The baseline count itself must be authenticated; it is not merely a producer's declaration.

Theorem 5.1 (Soundness and completeness of sparse replay). *Assume the underlying count proofs are sound for their bound inputs. The checks above accept exactly complete, correct incidence profiles presented in a valid minimal-extraction order. Every profile produced by Theorem 3.2 has such a certificate. The number of required subset values before reuse is at most*

$$V \leq s + \sum_{T \in \text{supp } h} |T|. \quad (9)$$

The verifier need not call the discovery algorithm, reproduce its block schedule, or perform a dense inverse transform.

Proof. Induct on the entry number. Before the first entry, all true coefficients are nonnegative. If all preceding entries are exact coefficients, subtracting them leaves the zeta transform of a nonnegative residual profile. The authenticated checks for entry j satisfy Lemma 3.1, so the next entry is also exact. After all entries, the residual has total mass zero; nonnegativity forces every omitted coefficient to vanish. This proves soundness. Discovery produces precisely these minimal positive residuals, so its entries satisfy the tests. Counting one query for each type and one for each of its members gives (9). None of these arguments relies on the producer’s particular deletion schedule. $\square$

5.2 Source binding and transport

The sparse certificate is not a bare list of numerical oracle answers. The maintained adapter stores one baseline local orbit proof, the sparse payload, a table of local coned proofs, and a sparse list mapping verification masks to proof indices. The checker rejects missing masks, repeated masks, unused proofs, a proof index assigned to two different physical unions, and distinct proof indices redundantly assigned to one physical union. The native local checker is imported from `fastunknot.interval_orbit_verify`, not from the producer.

The package includes exact tagged-hexadecimal JSON transport. It encodes integers in a distinguishable tagged object, preserving the distinction between an integer and a user string. The decoder restores Python integers before verification. It does not change Python’s global decimal conversion limit. In particular, masks and multiplicities with more than 16000 bits are not truncated or passed through floating point.

Proof size is real work. A small number of integer fields can each contain many bits, and an AHT trace can have many local records. All claims about certificate cost include the underlying trace lengths, not just the number of table entries. The algebraic verifier is not a denial-of-service sandbox; an application must cap serialized input size and provide its own cooperative deadline when receiving hostile certificates.

5.3 Two-pass proof construction

Discovery in the supplied adapter is untraced. A second pass records only the baseline and physical unions required by (8). This avoids retaining every exploratory trace, at the price of recomputing some counts. The global query and cycle allowances cover both passes. If proof production fails after successful discovery, the overall certified call is still INCONCLUSIVE; it does not expose a purported complete certificate assembled from a prefix.

A coarse upper bound is $Q + V + 2$ backend invocations, including the two baseline counts. Empty unions, repeated masks, and canonical-union reuse reduce it. A future single-pass producer could retain selected traces or cache richer verified states, but its memory and replay costs would need separate analysis. We do not use an unmeasured proof-caching strategy to improve the reported bound.

6 Why interval-encoded incidence has polynomial support

6.1 The classical weighted route

For arbitrary abstract nonnegative histograms, s can equal 2^r . Interval encoding imposes additional structure. Assign each point the vector

$$w(x) = (\mathbf{1}_{A_1}(x), \dots, \mathbf{1}_{A_r}(x)) \in \mathbb{N}^r. \quad (10)$$

This vector is constant between successive marked endpoints, hence on at most $2M + 1$ integer intervals. The weight of an orbit is the sum of its point weights, and its nonzero coordinate set is exactly (1). Thus a compressed list of orbit weights also gives the desired sparse histogram after equal supports are collected.

We use three classical facts from AHT's analysis [1]: the classical scheduler has at most $5k^2(2 + \log_2 N)$ cycles; weighted transfer preserves orbit sums and increases the number of live constant-weight intervals by at most four (Lemma 15 and its proof); and static intervals are archived as constant-weight interval records rather than individual points. These are imported ingredients, not new orbit-scheduler proofs.

It already follows from the weighted theorem that the sparse output has polynomial size. For clarity about the parameter dependence, we give an explicit, deliberately conservative accounting bound. Tracking both live and archived runs avoids a possible mistake: a bound on live runs alone does not automatically bound the cumulative output.

Theorem 6.1 (Explicit incidence-support bound). *For an interval system with parameters (N, k, M, r, B) as in Section 2, let*

$$L_0 = 1 + 5k^2(2 + B), \quad (11)$$

$$S_0 = 2M + 1 + (4k + 6)L_0. \quad (12)$$

Then

$$s \leq \min\{C, 2^r, S_0\} = O(M + (k + 1)^3(B + 1)). \quad (13)$$

For $N = 0$ the conclusion is immediate. The formula is an upper bound on occurring incidence types, not on the number of individual orbits.

Proof. The cases $N = 0$ and $r = 0$ are immediate, so assume both are positive. Run the classical weighted algorithm conceptually on (10). This run is used to prove a structural bound; the sparse implementation need not execute weighted transfer. Let R be the number of current constant-weight intervals plus the number of already archived constant-weight interval records. Initially $R \leq 2M + 1$. Normalize equal adjacent live runs after each stage; the archive may be normalized independently. Temporary endpoint refinement will be charged explicitly below.

At a static-contraction stage, the union of all pairing domains and ranges has at most $2k$ interval records, so its complement has at most $2k + 1$ static gaps. Refine the current constant-weight partition at the two endpoints of each gap. This increases the combined run count by at most $2(2k + 1) = 4k + 2$. Move the pieces contained in gaps from the live list to the archive and close the corresponding coordinate gaps. Moving an already refined piece changes its list, not the total number of pieces. Concatenation or merging decreases, rather than increases, that total.

Identity deletion, reflection trimming, periodic merger, and transmission leave the weight function unchanged. There is at most one weighted-transfer operation in a cycle, adding at most four live runs by the imported transfer bound. The following suffix truncation adds none: it deletes a suffix after its weight has been transferred. Therefore a complete cycle increases R by at most $(4k + 2) + 4 = 4k + 6$. The number of pairings never increases, so the initial k is a valid uniform bound throughout.

The classical cycle estimate, with an extra terminal cycle and with $B \geq \log_2 N$, gives at most L_0 cycles. The extra one also covers the unpaired case $k = 0$. At termination every original orbit is represented by one

archived point, but consecutive points with equal vector weight share an interval record. There are at most S_0 such records. Each record contributes one incidence signature, regardless of its length. Coalescing records with the same signature cannot increase their number. This proves $s \leq S_0$. The bounds $s \leq C$ and $s \leq 2^r$ are immediate. $\square$

Remark 6.2 (What the explicit bound does not say). The constant in (11) belongs to the classical AHT analysis. It is not a newly audited cycle constant for the maintained sharp Fine–Wilf scheduler. The structural conclusion concerns the original interval relation and is independent of which exact count backend is used by sparse recovery. The bound is intentionally loose, and no numerical optimality is claimed.

6.2 Consequences and attribution

Corollary 6.3 (No logarithmic port restriction for the local query). *With a polynomial-time exact unweighted interval-orbit backend, the full sparse incidence profile is computable in time polynomial in $k + M + r + B$, without assuming $r = O(\log N)$ or $r = O((\log N)^2)$. With polynomial-size independently replayable count proofs, the sparse profile also has a polynomial-size independently checkable certificate.*

Proof. Theorem 6.1 bounds s polynomially in the encoded parameters. Theorems 3.2 and 5.1 use at most $O(sr)$ discovery and verification values. Each coned query has at most $k + 2M$ pairings and the same universe size. Its input, computation, and optional proof therefore have polynomial size. The remaining operations manipulate polynomially many r -bit masks and $O(B)$ -bit counts. $\square$

The polynomial computability of weighted component information predates this work. The new implementation route is instead a reduction to the already certified *unweighted* kernel, coupled with a sparse proof format. It is a strict improvement over the inspected dense interface’s representation bound, not a claim to improve the asymptotic complexity class of the classical weighted problem.

The full-support static example in Section 11 does not contradict (13). Encoding all 2^r signatures using alternating interval marks requires many marked interval records. What would be exponentially large as a function of r alone need not be exponential as a function of the actual interval input length.

7 Bit costs, storage, and resource semantics

Let $P(k, B)$ be a monotone upper bound for a backend count call. For certified operation, enlarge P to cover local proof construction, the proof’s serialized bit length, and independent replay. Classical AHT supplies a polynomial count bound; a chosen implementation must separately meet the corresponding proof contract.

Define $D_s = \sum_{T \in \text{supp } h} |T|$ and let Q, V be the bounds in (7) and (9). A conservative bit-time bound for our direct subset-scanning implementation is

$$(Q + V + 2)P(k + 2M, B) + O\left((Q + V + s + 1)\left[r^2 + s(B + r + 1) + (M + 1)\log(M + 2)(B + r + 1)\right]\right), \quad (14)$$

up to polynomial input-validation and proof-serialization overhead. The displayed dictionary-cache accounting uses expected hash-table costs. Replacing it by a deterministic ordered dictionary adds comparison and logarithmic factors and preserves the polynomial conclusion. A simple linear scan of the polynomial-size cache also gives a deterministic polynomial bound without a hashing assumption.

Here is the accounting behind (14). An ordinary implementation scans port masks using r -bit arithmetic, safely bounded by $O(r^2)$ per query. A residual value checks at most s subset relations and subtracts $O(B)$ -bit multiplicities, costing $O(s(r + B + 1))$. Forming a selected physical union sorts at most M interval records

and compares B -bit endpoints. The verifier’s one-bit deletions and sparse table checks fit the same coarse bound. These operations are not assumed to be unit-cost arithmetic on arbitrarily large integers.

Ignoring backend traces, the sparse profile occupies $O(s(r + B + 1))$ bits. Cached physical unions occupy at most $O((Q + V)M(B + 1))$ bits in a straightforward representation. A two-pass certificate holds at most $V + 1$ distinct count proofs before physical-union reuse, together with its sparse index table. There is no hidden output array of length 2^r .

The result is polynomial after substituting (13), but it is not necessarily fast for all moderate-size inputs. The simple residual implementation can cost quadratic time in s across all probes. The deliberately loose structural bound can also be much larger than observed support. Output-sensitive accounting remains useful precisely because it does not replace actual s by that worst-case upper bound.

7.1 Three separate budgets

The implementation distinguishes support entries, backend invocations, and AHT cycles. A cycle allowance counts structural iterations, not all bit operations or elapsed time. A `check()` callback is forwarded to the maintained backend and polled during the sparse search. Input normalization and individual Python operations are not a hard real-time preemptive boundary, so a separate process-level policy is appropriate for adversarial inputs.

For a fixed encoded instance with all caps disabled, the preceding theorems apply. With a cap, no incomplete output is promoted to a complete profile. The native adapter shares its backend query and cycle allowances between ordinary discovery and optional proof production. Exhaustion publishes only status, reason, and work statistics. No local status is converted to UNKNOT or KNOTTED.

8 Incidence-resolved binary orientation data

An interval’s order reversal is not the same datum as a surface gluing’s orientation character. Let each supplied pairing g carry a separate parity $\epsilon_g \in \{0, 1\}$. On $X_N \times \{0, 1\}$ form the relation

$$(x, e) \sim (g(x), e + \epsilon_g \bmod 2). \quad (15)$$

This is encoded by $2k$ interval pairings on two consecutive blocks of length N . Lift every port to both sheets. There are at most $2M$ marked interval records and still r named ports.

A base component is *consistent* if its vertices admit binary labels whose differences equal the supplied edge parities. Equivalently, every closed walk has parity zero. This is a statement about the supplied signed relation. It becomes geometric orientability only when the parities have been derived and verified as the relevant orientation character.

Theorem 8.1 (Support-preserving orientation refinement). *Let h and $\tilde{h}$ be the base and lifted incidence histograms. Their supports are identical. If $o(T)$ and $u(T)$ are the numbers of consistent and inconsistent base components of signature T , then*

$$\tilde{h}(T) = 2o(T) + u(T), \quad h(T) = o(T) + u(T), \quad (16)$$

$$o(T) = \tilde{h}(T) - h(T), \quad u(T) = 2h(T) - \tilde{h}(T). \quad (17)$$

Once h is known, all coefficients of $\tilde{h}$ can be recovered using at most s additional lifted zeta values, plus one lifted baseline count.

Proof. A path in a base orbit lifts uniquely once its initial sheet is chosen. Every component upstairs therefore projects onto the entire base orbit. If all closed walks have parity zero, the two choices of initial sheet

remain disconnected and give two lifted components. If some closed walk has parity one, the two sheets are connected, giving one lifted component.

Every lifted component projects onto every point of its base orbit and the ports include both sheets. Thus every lift has exactly the same port signature as the base orbit. No new signature is introduced and none is lost. Counting one or two lifts gives (16) and solving the two equations gives (17).

Order the known support by nondecreasing cardinality, breaking ties arbitrarily. A proper subset always precedes its superset. For each known T compute

$$\tilde{h}(T) = \tilde{G}(T) - \sum_{\substack{S \in \text{supp } h \\ S \subsetneq T}} \tilde{h}(S). \quad (18)$$

Support equality proves there are no unlisted terms. One lifted zeta value per type suffices. The baseline checks the total; the necessary inequalities $h(T) \leq \tilde{h}(T) \leq 2h(T)$ are useful defensive checks, not a replacement for the exact cover construction. $\square$

The generic cover construction and known-support calculation are implemented and tested. A native source-bound certificate wrapper for the combined signed result is not included: it must authenticate the signed input, both-sheet port lift, and coned cover counts in addition to the base sparse certificate. The theorem shows that this wrapper needs no second support search. It does not license treating unauthenticated parity bits as a surface-orientability certificate.

Example 8.2 (A reflection with a fixed point). On five points, pair x with $4 - x$ and assign parity one. The two two-point orbits are consistent; the midpoint has a parity-one loop and is inconsistent. With one port containing every point, $h(\{1\}) = 3$ and $\tilde{h}(\{1\}) = 5$. Formula (17) gives two consistent and one inconsistent component. This case tests why a geometric reflection flag must not be substituted for a consistency conclusion.

9 Exact continuation operations and a sharp limitation

9.1 Coning without another orbit calculation

Once the complete profile is known, some later quotient operations can be performed directly on it. For $J \subseteq [r]$, cone all points of $\bigcup_{i \in J} A_i$ in the current relation.

Theorem 9.1 (Coning update). *Let $\mathcal{T}_J = \{T \in \text{supp } h : T \cap J \neq \emptyset\}$. If $\mathcal{T}_J$ is empty, the profile is unchanged. Otherwise remove all entries of $\mathcal{T}_J$ and insert the single entry*

$$\left(\bigcup_{T \in \mathcal{T}_J} T, 1 \right). \quad (19)$$

All other entries keep their multiplicities. The update is exact and does not increase the support size.

Proof. Every current component meeting the selected union is joined to every other such component. Its new port signature is the union of their old signatures. This describes exactly one new component even if a selected signature had enormous multiplicity. Untouched components remain separate and retain their signatures. The new signature intersects J , so it cannot coincide with an untouched signature. Replacing one or more selected types by one therefore does not increase support size. $\square$

A sequence of q such operations costs $O(qs)$ mask and count operations in a direct scan, or $O(qs(r+B+1))$ conservative bit work with fixed ports. The same theorem applies inductively after each quotient. No fresh AHT call is required for this restricted continuation.

9.2 Union re-marking and disjoint unions

Suppose each new port is a union of old ports, $B_j = \bigcup_{i \in J_j} A_i$. Map a signature by

$$\phi(T) = \{j : T \cap J_j \neq \emptyset\}. \quad (20)$$

The new profile is obtained by adding the multiplicities of types with equal $\phi(T)$. This follows directly from the existential definition of port incidence. Deleting or renaming ports is a special case. If two independent systems are combined disjointly and each named port is the disjoint union of its two old versions, their profiles add coefficientwise.

Corollary 9.2 (Restricted continuation congruence). *Two systems with the same incidence profile remain indistinguishable by any finite composition of coning selected port unions, union re-marking, and disjoint union with fixed auxiliary systems, when the observation is the resulting incidence profile.*

Proof. Each operation has a well-defined update depending only on the current profile. Induct on the number of operations. $\square$

This is a sufficient-state theorem for an explicit operation language, not a claim that incidence is a complete boundary state for a topological hierarchy. The distinction is essential for subsequent integration.

9.3 Why arbitrary attachments are not covered

Proposition 9.3 (Equal histograms can separate under an attachment). *There exist two interval-generated equivalence relations with identical port-incidence profiles for which adding the same singleton pairing gives different component counts.*

Proof. Use four points and a single port containing all four. The first relation has classes $\{0, 1\}, \{2, 3\}$; the second has classes $\{0, 2\}, \{1, 3\}$. Both have histogram $h(\{1\}) = 2$ and all other entries zero. Both relations are generated by singleton interval pairings. Adding $0 \sim 1$ changes nothing in the first system, but joins the two classes in the second. Their resulting component counts are two and one respectively. $\square$

An incidence histogram also cannot answer pointwise-intersection questions by substituting logical conjunction of ports. A component can meet A and B at different points even when $A \cap B = \emptyset$. More generally, it contains no point multiplicities, cyclic attachment orders, slopes, component Euler characteristics, embeddings, or individual attachment maps. Some of these can be obtained from richer weighted or geometric data; none can be silently reconstructed from a Boolean signature alone.

10 Implementation, trust boundaries, and integration

10.1 Package modules

The artifact uses only the Python standard library. The generic `sparse_ports/core.py` implements both deletion strategies, sparse certificate payloads, independent algebraic checking, coning, and union re-marking. `intervals.py` implements normalized physical unions, the coning oracle, and a bounded dense reference for controlled comparisons. `orientation.py` supplies the double cover and known-support refinement. `codec.py` handles tagged hexadecimal transport.

The opt-in `proveit.py` adapter imports the maintained orbit modules lazily, so the standalone research tests need no installation of ProveIt. It preserves the maintained convention that incomplete calculations expose no certified count. Its sparse histogram has an explicit encoding tag; a caller expecting the old dense array cannot accidentally treat a sparse pair list as the same API.

For example, with the maintained `fast` directory and this package on `PYTHONPATH`, a caller can use:

```
from fastunknot.interval_orbits import IntervalPairing
from sparse_ports.proveit import (
    analyze_port_incidence_sparse,
    verify_sparse_port_incidence_certificate,
)
pairs = [IntervalPairing(0, 3, 4, 7)]
ports = [[(0, 2)], [(1, 3)], [(7, 10)], []]
answer = analyze_port_incidence_sparse(
    10, pairs, ports, record_certificate=True,
)
if answer["status"] == "COMPLETE":
    assert verify_sparse_port_incidence_certificate(
        10, pairs, ports, answer["certificate"],
    )
```

This illustrates the intended integration call, not a claim that this exact native call was executed in the delivery environment.

10.2 The audit extraction is not the maintained implementation

The executed component experiments use `tests/aht_reference.py`, an explicitly labeled audit extraction of the inspected maintained unweighted scheduler. It keeps the interval reductions and sharp merger, but omits native proof recording and unrelated wrappers. It is not a byte-identical repository snapshot. Source attribution and the inspected blob identifier are retained in its header.

Small systems are checked against an independent point-expanded disjoint-set calculation. Huge static and periodic systems are checked against a non-expanding endpoint/residue calculation. The dense and sparse benchmark arms use exactly the same audit backend and physical-union cache, isolating the reconstruction strategy. The dense arm is a functional reference with the same dense-inversion architecture, not a byte-for-byte run of the maintained dense module.

Consequently these experiments support a local algorithmic comparison. They do not measure the maintained proof producer, certify its current installation, establish native normal-surface provenance, or show a whole-knot speedup. A claim of such a result would require a different measurement and validation boundary.

10.3 Proposed non-disruptive integration

Keep this work initially under a research subdirectory and expose the sparse interface alongside, not in place of, the dense function. Run `integration/native_smoke.py` in a real checkout. That script compares the current dense and sparse profiles on randomized systems, records and replays actual maintained local proofs, round-trips the certificate codec, and disables the producer while replaying. It is supplied but unexecuted here.

After those checks, add native normal-arc fixtures and paired query measurements. Only then consider promoting an opt-in sparse backend in the maintained module. Dense output should remain an explicit conversion with an explicit port limit, because producing every zero entry is intrinsically an output-size operation. Nothing in this package changes ordinary `recognize` dispatch or removes its complete fallback.

11 Validation and controlled component experiments

11.1 Executed checks

The included test log records 19 passing unittest methods. A method often checks many instances: the total is not merely 19 examples. The executed coverage includes all 6654 histograms with $r \in \{0, 1, 2, 3\}$ and coefficients in $\{0, 1, 2\}$, comparing both discovery strategies and certificate replay; 1000 random sparse abstract profiles; and 80 tests of the balanced-block bound on singleton supports. Large weights, empty ports, zero universes, malformed integers, callbacks, caps, and mutated certificates are included.

Five hundred random interval systems compare the audit AHT count with literal components and compare sparse, dense, and literal histograms. Three hundred further systems undergo five successive conings each, checking the profile update after every operation against literal quotient connectivity. Huge periodic cases use endpoint lengths up to roughly 4000 bits, and huge static and transport cases reach 16001-bit integers without point expansion. Five hundred random signed systems compare the refined profile with an independent breadth-first parity-consistency calculation.

The maintained-adaptor contract tests inject a backend with independent literal count witnesses. They test orchestration, exact proof bindings, shared budgets, codec round trips, and replay with the producer disabled. They are explicitly *not* execution of the maintained AHT trace producer/verifier. No statement about the repository’s full test count or Regina integration is inferred from this local test run.

These checks are evidence for implementation agreement, not substitutes for the proofs. In particular, finite tests cannot establish the structural support bound for every interval system. The latter depends on the classical transfer and cycle facts plus the accounting in Theorem 6.1.

11.2 Measurement protocol

The retained driver uses CPython 3.13.5 on the Linux platform recorded in `results/benchmarks.json`. The fixed shuffle seed is 261008505. Nine paired workloads have four arms: identical dense controls A and B, balanced splitting, and individual deletion. Each workload receives one warmup round and seven measured rounds, with arm order shuffled per round. Three larger capacity workloads run only the sparse arm. This gives 273 measured calls and 39 warmup calls. Correctness comparison is outside the timed region. Raw nanoseconds, query counts, all arms, and source hashes are retained, not just the favorable ratios.

The reported ratio is the median of seven within-round dense-A/sparse ratios, not the ratio of independently selected fastest runs. A value above one favors sparse recovery. Timing includes oracle creation and its baseline count, union preparation, all backend calls, and reconstruction. It excludes native local proof production and replay, which this benchmark does not execute. Timings are local measurements, not extrapolations to other hardware or an unknot corpus.

11.3 What the families test

For the coincident family, $N = 2^{500}$, there are no base pairings, and all r ports equal $[5, N - 7)$. The only signatures are the empty one, with multiplicity 12, and $[r]$, with multiplicity $N - 12$. Both dense and sparse methods use only two backend calls because of physical-union reuse. Their different time costs therefore isolate the dense mask/inversion overhead rather than faster orbit counting.

For the disjoint family, r separated two-point intervals mark otherwise static points. Its $r + 1$ occurring types are the empty signature and the r singleton signatures. Distinct port subsets have distinct physical unions, so the dense method cannot avoid exponentially many coned calls by union caching. Balanced extraction removes most absent combinations and also uses fewer logical probes than individual deletion on this family.

Workload	s	Dense A (ms)	Dense B (ms)	Split (ms)	Paired ratio
coincident-8	2	0.586	0.591	0.052	11.00
disjoint-8	9	28.011	27.187	3.682	7.40
coincident-10	2	2.824	2.838	0.070	40.17
disjoint-10	11	176.143	173.715	7.946	22.08
coincident-12	2	15.264	14.052	0.120	122.96
disjoint-12	13	1003.604	995.129	14.636	68.04
periodic-6	7	1.315	1.263	0.556	2.45
periodic-8	9	6.139	5.862	0.861	6.66
full-support-6	64	14.773	14.859	20.208	0.73

Table 1: Controlled component timings. Dense A and B are identical controls. The last column is the paired dense-A/split ratio; values below one are regressions. No native proof-generation or whole-knot time is included.

Workload	Dense calls	Split calls	Split requests	Linear requests
coincident-8	2	2	16	16
disjoint-8	256	21	57	72
coincident-10	2	2	20	20
disjoint-10	1024	26	79	110
coincident-12	2	2	24	24
disjoint-12	4096	33	101	156
periodic-6	37	15	49	42
periodic-8	114	21	71	72
full-support-6	64	64	606	384

Table 2: Backend calls and logical discovery requests on the same workloads. The dense reference evaluates all subset masks, but equal physical unions share backend work. Split requests can exceed distinct backend calls.

For the periodic family, a single pairing translates by $p = 2^{250} + 37$ on a universe of size $N = p(2^{250} + 1)$. Components are residue classes modulo p . Overlapping interval ports are checked independently by sweeping their residue endpoints. This family exercises actual compressed orbit reduction rather than a purely static universe, but it is still a controlled interval system, not an extracted knot hierarchy.

The full-support control uses $r = 6$, $N = 64$, no base pairings, and interval ports that encode the six binary digits of each point. All 64 signatures occur. Its interval encoding has 63 marked interval records; sparsity offers no output reduction. Balanced recovery is slower than dense inversion, approximately 20.2 ms versus 14.8 ms in the retained medians. The paired ratio is about 0.73. This negative control rules out a claim that the sparse method is uniformly faster.

On the sparse workloads, paired ratios range from approximately 2.45 to 123 in the retained data. These figures quantify this particular component-level ablation. They do not compare a fully optimized weighted AHT implementation against sparse coning, and do not show a speedup in ordinary recognition. Identical-control variability is visible in the raw samples; precision beyond the displayed few significant digits would not be justified.

11.4 Large-port capacity without a dense comparison

The capacity family uses $N = 2^{4000}$ and the same two-type coincident-port structure. Table 3 reports completed sparse calculations. No dense timing is attempted at 32, 128, or 1024 ports, and no speed ratio is

fabricated for those omitted runs.

Ports r	Endpoint bits	Support s	Orbit calls	Median (ms)
32	4001	2	2	0.298
128	4001	2	2	2.635
1024	4001	2	2	151.524

Table 3: Completed large-port sparse queries. These are capacity observations, not ratios against an executed dense arm. Endpoint size is binary length.

Only two backend count calls are needed at each size, but elapsed time still increases with r : constructing and comparing large masks and repeatedly forming unions is not free. Generic algebraic certificate-query accounting is also retained in the JSON, but is separate from these timed calls and from genuine local AHT trace costs. Quotienting provably identical ports before discovery could reduce this overhead, subject to a source-bound expansion map for the output and certificate.

12 Consequences for the research program

12.1 A local exponential factor is avoidable

The inspected dense representation spends 2^r time and output slots even when interval encoding permits only polynomially many occurring signatures. The sparse interface removes that representation penalty using the existing unweighted count boundary. Theorem 6.1 is stronger than an assumption that a particular benchmark happens to have small support: it makes the local polynomial statement uniform over interval-encoded instances.

This conclusion should not be advertised as an improvement from exponential to polynomial *unknot recognition*. Its unit of input is a supplied pairing system and marked interval list. A complete recognition procedure must still construct those systems, justify what they mean geometrically, choose successful decompositions, and bound all intermediate description sizes and the number of calls.

Proposition 12.1 (Conditional composition, with explicit obligations). *Suppose a sound and complete recognition procedure on an n -crossing input makes at most $n^{O((\log n)^a)}$ local incidence calls. Suppose every call has $k + M + r + B \leq n^{O((\log n)^b)}$, its geometric provenance is verified within the same kind of bound, and all other work has quasi-polynomial bit cost. Replacing dense incidence with the sparse exact interface contributes only quasi-polynomial time to that procedure.*

Proof. By Corollary 6.3, each call costs a fixed polynomial in its encoded parameter sum, hence at most $n^{O((\log n)^b)}$. Multiplying by the call bound remains quasi-polynomial. Add the separately bounded provenance and non-incidence work. Correctness requires the stated soundness and completeness hypotheses; it is not supplied by the time calculation. $\square$

The point is not that this elementary composition argument proves a new global complexity theorem. It identifies exactly which assumptions remain outside the local interface. The logarithmic-port hypothesis is no longer one of them. Bounds on M and B after repeated cutting, and a sufficient state for actual attachment maps, remain indispensable.

12.2 A precise correction to an interpretation of the old bound

The earlier 2^r estimate is correct for the maintained dense algorithm. It would be incorrect to interpret it as an intrinsic lower bound for computing all *occurring* interval-incidence types. The weighted AHT route

already prevents that interpretation; the present sparse reduction makes the distinction executable with the current unweighted proof interface. No change to the correctness of the old dense routine is alleged.

Likewise, equal signatures may justify batching certain aggregate queries, but they cannot generally justify merging components in a geometric state. Proposition 9.3 shows the missing information explicitly. Any future integration that discards attachment maps merely because two components have the same signature would need a stronger theorem than the one proved here.

13 Further research questions and concrete next experiments

The following questions are ordered roughly from immediate integration work to the broader hierarchy problem. Each asks for an additional result or measurement rather than treating a missing hypothesis as established.

1. Genuine native proof replay and differential testing. Run the supplied native integration script at a pinned maintained commit, then add adversarial mutations of every sparse proof binding and repeat against both classical and sharp merger proof versions. Does the same independent sparse verifier accept both without importing either scheduler? Retain the actual source hashes and the full native test log.

2. Native normal-arc and disc fixtures. Extract ports from actual validated normal-arc systems, rather than choosing coordinate intervals artificially. Measure the observed (k, M, r, B, s) and the distribution of signature sizes. Which geometric constructions produce small s and small D_s , and which fragment the port representation? A geometric port extractor should have its own independently replayable provenance contract.

3. A signed certificate adapter. Implement the source-bound double-cover wrapper suggested by Theorem 8.1. Prove and test that the checker binds every parity and every lifted port to the source, then compare known-support refinement against an independent second sparse search. The generic savings of one support search should be separated from actual trace costs.

4. One-pass weighted AHT versus sparse coning. Build or adapt a verified weighted transfer kernel and compare it fairly against this black-box reduction. The weighted algorithm can reuse more internal structure; sparse coning reuses an existing smaller trusted kernel. Which method minimizes bit operations, proof bytes, and replay time on actual normal-surface families? A hybrid should switch based on measured work, not merely on a port-count threshold.

5. Quotient identical or implication-related ports. Identical physical ports are safe to quotient with an explicit output expansion map. More generally, geometric containment $A_i \subseteq A_j$ forces every signature containing i to contain j . Can a canonical implication-poset representation reduce both search and certificate probes while preserving independent source checks? Numerical equality of coverage counts alone is not a sufficient justification.

6. Faster residual subset sums. The prototype scans all previously found types for each residual query. Develop a deterministic incremental data structure for $\sum_{T \subseteq U} h(T)$ restricted to the discovered support, with bit-cost bounds that improve this quadratic factor on geometrically occurring families. A generic dense zeta table would reintroduce the very cost being removed.

7. Sharpen the structural support estimate. The conservative bound $O(M + (k + 1)^3(B + 1))$ charges every possible static gap in every cycle. Can live/archive amortization remove a factor of k or weaken the dependence on B ? Determine matching lower-bound families for the number of distinct signatures, not merely for the number of components or distinct orbit weight vectors.

8. Optimal multi-support query complexity. The singleton lower bound does not settle the optimal reconstruction cost for arbitrary positive profiles. Characterize the best adaptive additive query bound in terms of r , s , signature sizes, and inclusion structure. Can a shared extraction tree avoid repeatedly discovering common mandatory ports? Any improvement must account for multiplicities rather than assuming unweighted support.

9. A richer exact continuation state. Find the smallest useful extension of the histogram that is sufficient for specific attachment maps arising in compressed cutting. One candidate is a sparse incidence profile together with a compressed boundary permutation or interval map. Prove a congruence theorem for the actual operation language, and exhibit separating examples when proposed state fields are omitted.

10. Port fragmentation under cutting. Even a polynomial local algorithm can be overwhelmed by an exponentially fragmented input. Establish bounds on the number of port intervals after one certified cutting operation and under a hierarchy. Can periodic or straight-line encodings replace long interval-union lists without destroying the support bound or the coning reduction?

11. Global hierarchy costs and completeness. Relate the encoded sizes of actual intermediate objects to the hierarchy length and pattern-complexity parameters. The target is a constructive bound on total bit work, including successful search, not just fast checking of a supplied hierarchy or its aggregate orbit data. This is the missing connection to a general quasi-polynomial recognizer.

12. Whole-pipeline evaluation and proof formalization. After native integration, measure complete recognition calls with paired controls, including adverse cases and certificate replay. Separately, formalize the nonnegative-residual and minimal-support certificate theorems; they form a compact first target because they do not require formalizing three-manifold topology. Only promote default behavior when correctness and full-call performance have both been established.

A A worked sparse certificate

Consider three ports and the profile

$$h(\emptyset) = 2, \quad h(\{1\}) = 3, \quad h(\{2, 3\}) = 5, \quad h(\{1, 2, 3\}) = 7,$$

with all other coefficients zero. The baseline is $C = 17$. The order $\emptyset, \{1\}, \{2, 3\}, \{1, 2, 3\}$ is a valid minimal-extraction order. For the empty type, the single check is $G(\emptyset) = 2$. For $\{1\}$, $G(\{1\}) = 5$ and subtracting the earlier weight two gives three; deleting port one leaves residual zero.

For $\{2, 3\}$, $G(\{2, 3\}) = 7$, while $G(\{2\}) = G(\{3\}) = 2$. Subtracting the previously recovered empty type leaves five at the pair and zero at both deletions. For the full type, $G(\{1, 2, 3\}) = 17$; the three earlier weights sum to ten. Its deletion values are $G(\{2, 3\}) = 7$ and $G(\{1, 2\}) = G(\{1, 3\}) = 5$, exactly exhausted by earlier contained types. The residual at the full type is seven and each deletion residual is zero. Finally $2 + 3 + 5 + 7 = 17$ proves no omitted positive coefficient.

This example distinguishes the independent certificate from a log of the search. The verifier does not need to know which balanced blocks were visited, which candidate unions were cached, or which failed probes the producer made before finding these types. It checks only the local minimality conditions and total mass.

B Reproduction and artifact inventory

From the package root, the standalone tests and retained-data tables are reproducible with:

```
python -m unittest discover -s tests -v
python benchmarks/make_tables.py
cd article
sh build.sh
```

To obtain new timing samples without overwriting the retained experiment:

```
python benchmarks/run_benchmarks.py \
  --output results/benchmarks_rerun.json
```

The driver and article-table generator deliberately use different filenames for an unreviewed rerun and the retained evidence. To execute genuine native integration in an available checkout:

```
python integration/native_smoke.py /path/to/ProveIt --cases 250
```

This last command is a required next validation step, not a completed experiment in the supplied test log.

The archive contains the TeX source, compiled PDF, generated table fragments, all research modules and tests, the audit-only kernel with attribution, raw benchmark data, test log, source provenance, a claim-status ledger, build/reproduction scripts, and integration guidance. It contains no redistributed third-party research PDFs or font files. Its code is offered under MIT-0 for compatibility with the inspected project.

References

- [1] I. Agol, J. Hass, and W. Thurston, *The computational complexity of knot genus and spanning area*, Transactions of the American Mathematical Society **358** (2006), 3821–3850. See Theorem 12, Section 6, Lemma 15, and Theorem 16. [arXiv:math/0205057](#).
- [2] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, 2026 preprint, version 1. See Section 9. [arXiv:2607.23350](#).
- [3] N. H. Bshouty and H. Mazzawi, *Optimal query complexity for reconstructing hypergraphs*, 2010 preprint. [arXiv:1001.0405](#).
- [4] E. Price and J. Scarlett, *A fast binary splitting approach to non-adaptive group testing*, APPROX/RANDOM 2020, Leibniz International Proceedings in Informatics, Article 13. [doi:10.4230/LIPIcs.APPROX/RANDOM.2020.13](#). [arXiv:2006.10268](#).
- [5] V. Reshetnikov, ProveIt repository, *UnknotRecognition: interval incidence and independently replayable interval orbits*. Inspected modules `fast/fastunknot/interval_incidence.py`, `interval_orbits.py`, and `interval_orbit_verify.py`. Incidence integration baseline: commit `0441aad3967b69621edabfdb85367d465b4fb32f` (8 October 2026). [Pinned source directory](#). Individual inspected blob hashes are in `provenance.json`.
- [6] ProveIt, *Independent orbit proofs and reusable marked-union queries*, `synthesis/orbit_certificates.tex`, inspected blob `a83b3ab44d95b2e8c35752266d9db626ca126225`. [Maintained article section](#). Accessed 8 October 2026; the moving link is not a claim of immutable content.